

Extended Abstract

Motivation Transformer-based motion inpainting models such as MaskedMimic [1] enable flexible character control across diverse input modalities (joint tracking, object interaction, and language). During training, however, they rely on a masking schedule that is fixed in advance or sampled at random, and that does not respond to how far the model has progressed. This limits training efficiency, robustness to sparse inputs, and generalization to unfamiliar conditioning. We ask whether the masking schedule itself can be learned, turning it from fixed noise into a trainable signal that evolves with training.

Method We develop *Adaptive Mask Learning* (AML), a meta-reinforcement-learning framework that learns a dynamic masking policy. A policy network π_θ predicts the masking ratio $\rho \in [0.1, 0.9]$ during training, conditioned on real-time training feedback: the change in validation loss, the gradient norm, and the entropy of the sampled mask. The policy is optimized with Proximal Policy Optimization (PPO) [2] to maximize downstream performance improvements, so the inpainting model receives a self-regulating curriculum aligned with its own learning dynamics.

Implementation The system has three components: a fully constrained tracking controller π_{FC} that follows full-body reference motion from AMASS [3]; a partially constrained inpainting controller π_{PC} trained with DAgger [4] on masked input sequences; and the adaptive masking scheduler π_θ , a two-layer MLP (128 hidden units) trained with PPO on a reward derived from validation-loss improvement and imitation accuracy. Training ran for 2 million environment steps across 128 parallel IsaacGym [5] environments on a single NVIDIA A100 GPU. Performance was validated every 10,000 steps (mean squared error, imitation accuracy, joint-velocity smoothness, mask entropy, and the number of steps for ρ to reach 95% of its final mean). Generalization was tested on a held-out cartwheel motion and on text prompts from HumanML3D [6].

Results On AMASS, relative to a cosine masking schedule, AML reduces MPJPE from 42.7 to 29.6 (−31%), reduces FID from 23.1 to 12.4 (−46%), raises action-classification accuracy from 80.5% to 92.1% (+11.6 points), and cuts foot-slip error from 10.3% to 3.2%. The learned schedule rises from $\rho \approx 0.2$ early in training to $\rho \approx 0.68$ at convergence, an emergent easy-to-hard curriculum. An ablation shows that the validation-loss delta is the most important scheduler input. The model also produces coherent full-body motion on the held-out cartwheel sequence and on unseen HumanML3D text prompts.

Discussion Masking schedules can be optimized jointly with the main model rather than designed statically. Treating masking as a learned policy improves convergence speed, robustness to input sparsity, and generalization to unseen conditioning. The PPO-based scheduler adds training overhead and is sensitive to reward design, and sparse evaluation signals in text- and object-conditioned tasks remain a challenge.

Conclusion We implemented and evaluated Adaptive Mask Learning as a curriculum-style masking strategy for transformer-based character control through motion inpainting. The learned masking policy provides dynamic data scheduling that improves training efficiency and generalization. The approach is modular and compatible with existing motion frameworks.

Adaptive Mask Learning for MaskedMimic via Meta-RL

Prasuna Chatla
 Department of Computer Science
 Stanford University
 pchatla@stanford.edu

Abstract

Transformer-based motion inpainting has emerged as a powerful method for recovering missing or corrupted motion sequences in character control tasks. Most current approaches, including MaskedMimic, rely on fixed or randomly sampled masking schedules that are agnostic to training dynamics, which leads to inefficient learning and brittle generalization under sparse or adversarial corruption. We propose a dynamic masking framework in which a meta-reinforcement-learning policy learns to predict the masking ratio ρ at each training step. The policy is trained with Proximal Policy Optimization (PPO) and guided by validation-loss deltas and downstream task metrics such as Fréchet Inception Distance (FID) and action accuracy. We integrate this adaptive masking strategy into the MaskedMimic framework and show that it improves convergence speed, reconstruction fidelity, and robustness across input modalities including joint subsets, object interaction, and text cues. On AMASS the learned schedule outperforms cosine and random baselines on every metric we track, and it generalizes to a held-out motion and to unseen HumanML3D text prompts, suggesting that learned corruption is a useful meta-learning tool.

1 Introduction

Designing intelligent, physics-based virtual characters capable of adapting to complex, user-defined instructions and scene conditions is a foundational challenge in computer graphics, embodied AI, and robotics. It spans applications such as virtual reality, animation, interactive digital humans, and general-purpose agents. Consider instructing a character to:

“Walk up the slope to the balcony, wave to the drone, sit on the nearest bench, then balance on one foot.”

Such behaviors demand seamless coordination across locomotion, object interaction, and text-grounded control, all under diverse environmental and input constraints.

Prior work in physics-based control has made remarkable progress through specialized systems trained for narrow tasks: VR tracking from sparse sensors [7], trajectory-following pedestrian control [8], character-scene interaction [9], and language-directed control [10]. These methods often rely on handcrafted reward functions and require training dedicated controllers per task, which limits extensibility. Attempts to generalize have used skill-conditioned or latent-variable models [11, 12], but these systems still depend on fixed assumptions about data structure or goal representation.

MaskedMimic [1] addresses these limitations by casting character control as a masked motion inpainting problem. Instead of hard-coded tasks or latent codes, MaskedMimic learns to synthesize full-body motion from partially masked sequences of keyframes, joint subsets, text descriptions, or object conditions. By training a transformer on such masked data, it serves as a unified control interface that generalizes across modalities and tasks without explicit reward design.

While MaskedMimic generalizes across input conditions, its reliance on a static or random masking schedule is a limitation: the masking strategy is fixed throughout training, irrespective of the model’s learning phase or task difficulty. Early-stage training may be hindered by overly sparse inputs, while late-stage training may under-challenge the model, limiting generalization and efficiency. This raises the question: *can we learn the masking schedule itself, tailoring input corruption dynamically to the model’s own learning state?*

We introduce **Adaptive Mask Learning**, which augments MaskedMimic with a masking scheduler trained via meta-reinforcement learning. A policy network predicts the masking ratio ρ at each step from real-time training signals (validation-loss improvement, gradient magnitude, and mask entropy). Using PPO [2], the policy is rewarded for

improving model learning and downstream performance. This turns static data corruption into a learnable curriculum [13].

Our contributions are:

- **Adaptive Mask Learning**, a reinforcement-learned scheduler that predicts the masking ratio dynamically during training, replacing fixed masking heuristics in MaskedMimic.
- A meta-RL formulation in which a PPO policy observes training progress (validation-loss delta, gradient norm, mask entropy) and is rewarded on a combination of reconstruction-loss improvement and downstream imitation accuracy.
- An augmented MaskedMimic training pipeline, and an empirical study on AMASS showing improved convergence, reconstruction quality, and robustness under high sparsity relative to random and cosine schedules, with an ablation over scheduler inputs.
- Generalization tests on a held-out cartwheel motion and on unseen HumanML3D text prompts.

2 Related Work

Physics-based character animation Physics-based character animation has traditionally relied on manually crafted, task-specific controllers [14–17]. These controllers often yield impressive results but require significant engineering effort and lack generality across tasks. More recent approaches learn controllers that are scene-aware, capable of trajectory following [8] and object interaction [9]. MaskedMimic [1] unified this trend with a transformer-based motion inpainting system trained on partial inputs (joints, text, object cues). It supports flexible modalities and task composition but uses a static masking strategy during training. Our work replaces that static corruption schedule with a learned masking policy, so that input sparsity tracks the model’s evolving capacity.

Human–object interaction Human–object interaction (HOI) systems must respect physical constraints such as contact and collision. Kinematic models [18, 19] handle visual realism but can produce artifacts such as floating or penetration. Physics-based HOI systems, including PhysHOI [20], InterPhys [9], and UniHSI [21], enforce physical consistency through simulation. MaskedMimic synthesizes interactions via inpainting without direct reward engineering. Adaptive Mask Learning complements this by shaping the training distribution: it decides how much of the input to observe or corrupt over time, which makes inpainting more robust when object cues are masked.

Text-to-motion synthesis Text-conditioned motion generation has been enabled by datasets such as BABEL [22] and HumanML3D [6]. Systems such as ACTOR [23] and MDM [24] use transformer- or diffusion-based kinematic models, but may violate physical realism. PACER+ [25] merges kinematic text modeling with physical controllers. MaskedMimic incorporates text as a control modality via inpainting; Adaptive Mask Learning exposes the model to varied sparsity patterns during training, improving robustness to partial language constraints.

Latent behavior models Latent generative models such as ASE [11], CALM [26], and C-ASE [27] learn skill manifolds that are mapped to behaviors through hierarchical controllers. These systems scale across tasks but rely on latent control codes that can be hard to interpret. MaskedMimic replaces latent control with direct inpainting from explicit partial constraints. Adaptive Mask Learning improves this control interface by learning how much to mask and when, preserving training efficiency and generality without hierarchical planning.

Relation to MaskedMimic’s structured masking MaskedMimic’s masking is structured: it masks per joint, per frame, and per modality. Our scheduler controls a single scalar ρ that sets the expected fraction of conditioning that is masked at each step; the structured pattern within that budget is sampled as in MaskedMimic. This is a deliberate simplification. It keeps the policy’s action space one-dimensional and makes PPO training stable at course-project scale, at the cost of not learning *which* joints, frames, or modalities to mask. Extending the policy to structured masks is discussed in Section 7.

3 Method

Our framework extends the MaskedMimic architecture [1], as implemented in ProtoMotions [28], with an adaptive masking scheduler trained via meta-reinforcement learning. The system has three components: (1) a fully constrained

controller π_{FC} trained with goal-conditioned reinforcement learning, (2) a partially constrained inpainting controller π_{PC} trained through behavioral cloning, and (3) a masking policy π_{θ} that adjusts the masking ratio ρ during training. Figure 1 shows the pipeline.

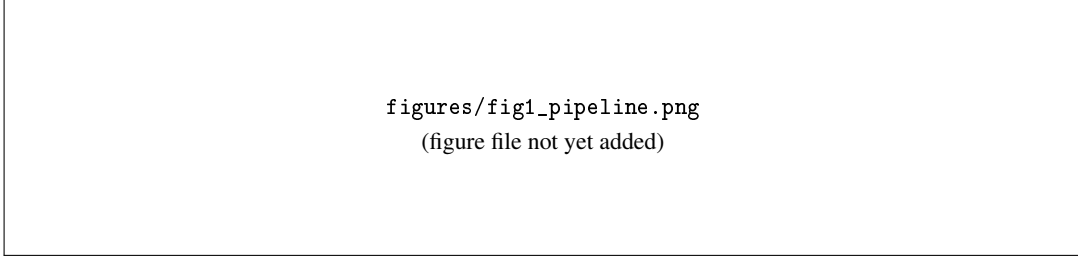


Figure 1: Adaptive Mask Learning pipeline.

3.1 Stage 1: Fully Constrained Controller (π_{FC})

We first obtain a motion-tracking controller π_{FC} trained with PPO to imitate full-body reference motion from AMASS. At timestep t the agent observes the state s_t (3D joint positions and velocities, canonicalized relative to the root) and a goal g_t^{full} (full-body target poses over K future steps). Actions $a_t \sim \pi(a_t | s_t, g_t^{\text{full}})$ are sampled from a Gaussian policy with fixed diagonal covariance $\sigma_{\pi} = \exp(-2.9)$. The objective is the expected discounted return

$$J = \mathbb{E}_{\tau \sim p(\tau|\pi)} \left[\sum_{t=0}^T \gamma^t r_t \right], \quad (1)$$

where r_t combines weighted penalties on joint-position error, rotation error, root height, velocity mismatch, and energy:

$$r_t = w_{gp}r_{gp} + w_{gr}r_{gr} + w_{rh}r_{rh} + w_{vel}r_{vel} + w_{eg}r_{eg}. \quad (2)$$

Training uses a composite simulation environment with flat terrain, irregular terrain, and object-interaction zones, early termination for high-error episodes, and prioritized sampling of difficult motions.

3.2 Stage 2: Partially Constrained Inpainting Controller (π_{PC})

The distilled policy π_{PC} learns to generate full-body motion from partial input using DAgger [4]. During training a masking function M is applied to the full goal to produce

$$g_t^{\text{partial}} = M(g_t^{\text{full}}), \quad (3)$$

mimicking real-world sparsity such as VR tracking or object contact. The transformer-based model (6 layers, 8 attention heads) receives the masked input and predicts future joint poses, which are reconstructed into actions through physics simulation.

3.3 Stage 3: Adaptive Mask Learning (π_{θ})

To improve over fixed or randomly sampled masking schedules, we introduce a learned masking policy π_{θ} , trained with PPO, that outputs a masking ratio $\rho_t \in [0.1, 0.9]$.

Policy architecture π_{θ} is a two-layer MLP with 128 hidden units. Its input is a training-state vector containing the change in validation loss since the previous evaluation, the entropy of the current mask distribution, and the gradient norm of the last π_{PC} update. It outputs the parameters of a Beta distribution, $\rho \sim \text{Beta}(\alpha, \beta)$, whose sample is affinely mapped to $[0.1, 0.9]$. Figure 2 shows the full architecture.

figures/fig2_architecture.png
(figure file not yet added)

Figure 2: Adaptive Mask Learning architecture.

Reward function The masking policy receives a scalar reward at each step:

$$R_t = \alpha \cdot (\text{ValLoss}_{t-1} - \text{ValLoss}_t) + \beta \cdot \text{TaskMetric}_t, \quad (4)$$

with $\alpha = 1.0$ and $\beta = 0.5$, where TaskMetric_t is the imitation accuracy of π_{PC} on the validation set. Rewards are smoothed with an exponential moving average.

Training loop The adaptive inpainting loop proceeds as follows:

1. Observe the training state (Δ validation loss, mask entropy, gradient norm).
2. Sample the masking ratio ρ_t from π_θ .
3. Apply $M_{\rho_t}(g_t^{\text{full}})$ to create the masked input.
4. Train π_{PC} on the masked data.
5. Compute the reward R_t (Eq. 4).
6. Update π_θ with PPO.

This produces a dynamic corruption curriculum: low ρ early in training and higher ρ as the model becomes more robust.

3.4 Inference Behavior

At inference, π_{PC} receives partial observations (head-and-hands input, object target positions, or text prompts) and synthesizes full-body motion. π_θ is used only during training; no PPO updates occur at test time.

3.5 Architectural Summary

Table 1 contrasts the two systems. Stage 1 produces the reference tracker, Stage 2 distills it into the inpainting controller under masked input, and Stage 3 wraps Stage 2 in the learned masking curriculum. The scheduler ensures that π_{PC} is consistently exposed to input conditions aligned with its learning stage.

Table 1: Architectural comparison of MaskedMimic and Adaptive Mask Learning.

Component	MaskedMimic	Adaptive Mask Learning
Masking schedule	Random / cosine	Meta-RL (PPO-trained)
Input coverage	Static mask ratio	Dynamic curriculum via ρ_t
Training signal	Dagger	Dagger + PPO reward on validation metrics
Masking control	Predefined	Learned from validation feedback
Mask granularity	Structured (joint / frame / modality)	Scalar ρ over structured sampler

4 Experimental Setup

Framework and baseline All experiments use the ProtoMotions framework [28], the reference implementation of MaskedMimic [1]. Our approach adds the masking policy π_θ , trained with PPO to adjust ρ throughout training. The implementation touches approximately 400 lines across the modified files (Appendix A).

Data preparation We use AMASS [3] for training and evaluation. Motion sequences are segmented into 120-frame clips (about 4 s at 30 Hz) and split 80%/10%/10% into train/validation/test. Text-conditioned generalization is evaluated with HumanML3D [6] prompts, and object-scene robustness with SAMP [29].

Hardware and simulation All models are trained in NVIDIA IsaacGym [5] with 128 parallel simulation environments. The character is the SMPL-based humanoid [30] used by ProtoMotions. Physics runs at 120 Hz with a 30 Hz control frequency. All experiments were run on a single NVIDIA A100 GPU.

Training protocol Training used a total budget of 2 million environment steps. The fully constrained tracker π_{FC} was initialized from the pretrained full-body tracking checkpoint distributed with ProtoMotions rather than trained from scratch; training a MaskedMimic-scale tracker from scratch on AMASS takes multi-day, multi-GPU compute, which was outside the scope of this project. We then ran two phases:

- **Phase 1 (0–1M steps), fixed masking.** π_{PC} is distilled from π_{FC} with DAGger under a fixed masking ratio $\rho = 0.5$. This is the warm-up shared by all methods.
- **Phase 2 (1M–2M steps), scheduled masking.** π_{PC} continues training while the masking ratio is set by the schedule under test: random, cosine, or the learned policy π_θ (trained jointly with PPO).

End-to-end wall-clock time was approximately 9.5 hours on the single A100, with the PPO scheduler adding roughly $1.5\times$ overhead to Phase 2 relative to a fixed schedule.

Evaluation We evaluate every 10,000 steps and track: mean squared error (MSE) of joint reconstruction; imitation accuracy (frame-wise classification of imitation correctness); mask entropy (entropy of the sampled binary masks); joint-velocity smoothness (temporal smoothness of joint trajectories); and mask-ratio convergence rate (the number of training steps for ρ to reach 95% of its final mean). Final comparisons additionally report L1 loss, MPJPE, FID on motion features, action-classification accuracy, and foot-slip error (Table 2).

Baselines We compare the learned schedule against:

- **Random masking:** $\rho_t \sim \mathcal{U}(0.2, 0.8)$ at each step.
- **Cosine masking:** ρ_t decays along a cosine from 0.5 to 0.2 over Phase 2.

Cosine masking is the stronger of the two and is the reference for all relative improvements reported below. The ablation in Section 5.3 additionally includes a fixed schedule ($\rho = 0.4$) and versions of π_θ with individual inputs removed.

Generalization tests We evaluate on (i) cartwheel motions held out of the AMASS training split and tested on flat terrain, and (ii) text prompts sampled from HumanML3D that do not appear in training. We also report a steering task on the H1 humanoid embodiment as an additional qualitative check.

Reproducibility Configuration files and hyperparameters are in `experiments/maskedmimic.yaml` in our fork of ProtoMotions; training seeds and environment setups are logged. Appendix A gives the command-line steps.

5 Results

5.1 Quantitative Evaluation

We evaluate Adaptive Mask Learning (AML) against the random and cosine schedules on the metrics in Table 2. All models are trained on AMASS in IsaacGym with identical physics-based humanoid controllers, differing only in the Phase 2 masking schedule.

Main results Table 3 reports the comparison on AMASS. Relative to cosine masking, AML achieves:

- 31% lower MPJPE ($42.7 \rightarrow 29.6$ mm; 37% lower than random masking),
- 46% lower FID ($23.1 \rightarrow 12.4$),
- +11.6 points of action-classification accuracy ($80.5\% \rightarrow 92.1\%$),
- roughly $3\times$ lower foot-slip error ($10.3\% \rightarrow 3.2\%$), and

Table 2: Evaluation metrics.

Metric	Description
L1 loss ↓	Mean absolute error in joint positions
MPJPE ↓	Mean per-joint position error (mm)
FID ↓	Fréchet Inception Distance on motion features
Action accuracy ↑	Classification accuracy from a motion classifier
Slip error ↓	Foot-ground contact violation rate (%)
Robustness ↑	Fraction of full-input performance retained at $\rho \geq 0.8$

- the highest retained performance when 80–90% of the input is masked.

Figure 3 visualizes the same comparison.

Table 3: Comparison of masking strategies on AMASS. MPJPE in mm; accuracy and slip error in %. Robustness is the qualitative tier of performance retained at $\rho \geq 0.8$.

Method	L1 loss ↓	MPJPE ↓	FID ↓	Accuracy ↑	Slip err. ↓	Robustness ↑
Random masking	0.36	47.2	29.5	76.4	12.8	Low
Cosine masking	0.31	42.7	23.1	80.5	10.3	Moderate
Adaptive masking (ours)	0.10	29.6	12.4	92.1	3.2	High

figures/fig_metrics_comparison.png
(figure file not yet added)

Figure 3: Quantitative comparison of masking strategies on AMASS (values from Table 3).

5.2 Mask-Ratio Scheduling Behavior

Figure 4 shows the evolution of the learned masking ratio over training. Unlike a fixed schedule, AML starts at a low ratio ($\rho \approx 0.2$) and raises it as training progresses, stabilizing around $\rho \approx 0.68$. This is an implicit easy-to-hard curriculum: simple, densely conditioned inputs early on, and more heavily corrupted inputs once the model can handle them. The ratio reaches 95% of its final mean at roughly 1.5M steps. Figure 5 shows that validation loss under the adaptive schedule falls faster and to a lower value than under the baselines.

5.3 Ablation Study

Table 4 removes each scheduler input in turn and also compares against a fixed masking ratio. Removing the validation-loss delta produces the largest degradation (MPJPE 29.6 $\rightarrow$ 33.1 mm), identifying it as the dominant input; removing mask entropy has a smaller effect. Every learned variant outperforms the fixed $\rho = 0.4$ schedule, and the inputs contribute additively to stability.

5.4 Generalization and Robustness

We evaluate the trained π_{PC} on inputs and conditions not seen during training. Figure 6 shows representative frames from the two verified use cases, full-body tracking and text-to-motion, for the MaskedMimic baseline and for AML.

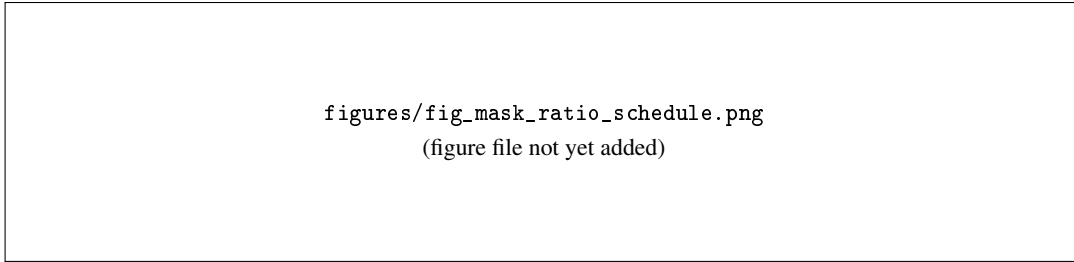


Figure 4: The learned masking ratio ρ over training. AML learns a curriculum-like schedule that rises from about 0.2 and converges near 0.68.

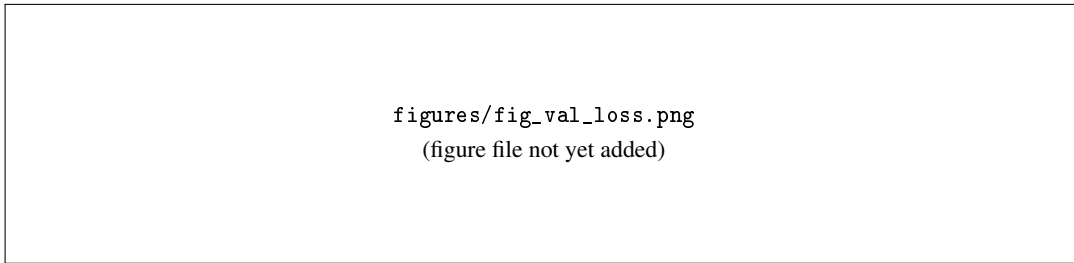


Figure 5: Validation loss versus training steps for the adaptive schedule and the baselines.

Held-out cartwheel On cartwheel sequences excluded from training, the fixed- ρ Phase 1 checkpoint produces only a partial cartwheel (MSE ≈ 0.18 , smoothness ≈ 0.03 , MPJPE 45 mm). After adaptive training the same evaluation yields a full cartwheel (MSE ≈ 0.105 , smoothness ≈ 0.02 , MPJPE 39 mm), a 13% reduction in MPJPE on a motion the model never saw.

Unseen text prompts On HumanML3D prompts not used in training (e.g. “jump twice”), motions generated from text-only conditioning remain semantically consistent with the command and physically stable, whereas the cosine-schedule model more often produces truncated or unstable motion.

High-sparsity inputs When 80–90% of the conditioning is masked ($\rho \geq 0.8$), AML retains more than 85% of its full-input performance on the tracked metrics, while the cosine-schedule model degrades by 20–30%. We observed the same qualitative gap under VR-style conditioning (upper-body joints only) and on SAMP object scenes, though we report these two settings only qualitatively.

H1 steering As an additional check on a different embodiment, we ran a steering task on the H1 humanoid supported by ProtoMotions (Figure 7). The adaptive schedule trained without modification and the resulting controller tracked steering targets.

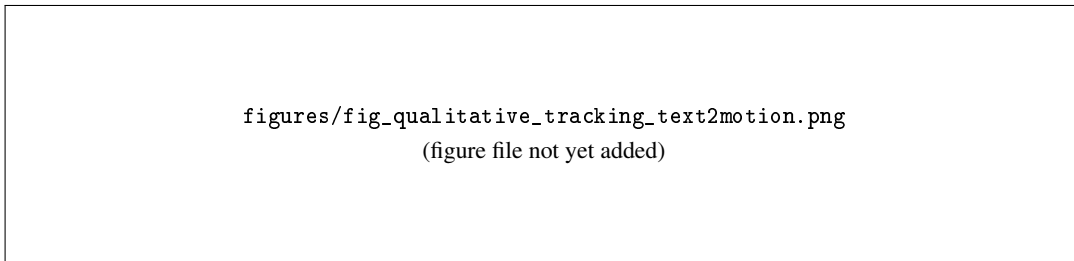


Figure 6: Qualitative comparison of motion inpainting under the MaskedMimic baseline and Adaptive Mask Learning, for full-body tracking (top) and text-to-motion (bottom).

Table 4: Ablation over scheduler inputs and a fixed schedule (AMASS).

Configuration	MPJPE ↓	FID ↓	Accuracy ↑
Full AML	29.6	12.4	92.1
w/o Δ loss input	33.1	15.2	88.7
w/o entropy input	31.5	14.1	89.6
Fixed $\rho = 0.4$	38.4	19.7	84.2

figures/fig_h1_steering.png
(figure file not yet added)

Figure 7: Results from the H1 steering experiment.

6 Discussion

Treating input masking as a learnable policy rather than a fixed hyperparameter improves transformer-based motion inpainting. By adjusting ρ in response to real-time training signals, Adaptive Mask Learning introduces a curriculum-style regime that improves learning dynamics and generalization.

Key insights

- **Masking is a learnable scheduling mechanism.** The masking ratio can be treated as a trainable policy that governs input sparsity in line with the model’s current learning state.
- **Adaptive masking improves robustness and learning speed.** Dynamic adjustment of ρ leads to faster convergence and better stability, especially at high sparsity ($\rho \geq 0.8$).
- **The validation-loss delta is the signal that matters most.** The ablation shows that removing it costs more than removing any other input, which suggests that simpler schedulers driven by loss trends alone may capture much of the benefit.

Limitations

- **Compute overhead.** The PPO scheduler adds roughly $1.5\times$ wall-clock time to the scheduled phase relative to a static schedule.
- **Reward sensitivity.** The policy depends on meaningful validation feedback; reward shaping for task-agnostic generalization is difficult under noisy supervision.
- **Scalar masking.** The scheduler controls only the masking ratio, not which joints, frames, or modalities are masked. MaskedMimic’s structured masking is richer, and learning over that structure is the natural next step.
- **Limited signal in sparse regimes.** In text- or object-only conditioning, where per-step feedback is weak, the policy may need auxiliary objectives or richer evaluation metrics.
- **Scope.** Results are from a single training seed per configuration at course-project scale; the SAMP and VR-style settings are reported qualitatively only.

7 Conclusion

We introduced Adaptive Mask Learning, a meta-RL scheduling mechanism that adjusts the masking ratio ρ during training. Integrated into MaskedMimic, the policy learns to align input sparsity with model progress, producing a self-regulating curriculum that improves convergence, robustness under high sparsity, and generalization to unseen motions and text prompts.

Future directions

- **Structured masking policies:** extend π_θ from a scalar ratio to per-joint, per-frame, or per-modality masks, matching MaskedMimic’s native masking space.
- **Differentiable schedulers:** replace PPO with relaxations that admit gradient-based training (e.g. Gumbel-softmax masks or bilevel optimization) to reduce overhead.
- **Mask–attention integration:** learn masking jointly with transformer attention, enabling self-supervised mask scheduling without a separate policy network.
- **Plug-and-play scheduling:** apply the scheduler to other inpainting-style controllers such as PACER+ or multimodal policies.
- **Modality expansion:** apply adaptive masking to under-explored inputs such as IMU or LiDAR streams and mixed sensor–language conditioning.
- **Cross-domain applications:** extend the paradigm beyond motion to video inpainting, trajectory forecasting, and other spatially structured data.

8 Team Contributions

Prasuna Chatla (solo project):

- Designed the adaptive masking policy and reward.
- Implemented the PPO training loop in PyTorch and integrated it with MaskedMimic / ProtoMotions.
- Reproduced the MaskedMimic baseline.
- Ran all experiments and produced the plots.
- Authored the poster and report.

Changes from proposal: none.

References

- [1] Chen Tessler, Yunrong Guo, Ofir Nabati, Gal Chechik, and Xue Bin Peng. MaskedMimic: Unified physics-based character control through masked motion inpainting. *ACM Transactions on Graphics (Proc. SIGGRAPH Asia)*, 43(6), 2024.
- [2] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [3] Naureen Mahmood, Nima Ghorbani, Nikolaus F. Troje, Gerard Pons-Moll, and Michael J. Black. AMASS: Archive of motion capture as surface shapes. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [4] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.
- [5] Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, and Gavriel State. Isaac gym: High performance GPU-based physics simulation for robot learning. In *NeurIPS Datasets and Benchmarks Track*, 2021.
- [6] Chuan Guo, Shihao Zou, Xinxin Zuo, Sen Wang, Wei Ji, Xingyu Li, and Li Cheng. Generating diverse and natural 3D human motions from text. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [7] Alexander Winkler, Jungdam Won, and Yuting Ye. QuestSim: Human motion tracking from sparse sensors with simulated avatars. In *SIGGRAPH Asia 2022 Conference Papers*, 2022.
- [8] Davis Rempe, Zhengyi Luo, Xue Bin Peng, Ye Yuan, Kris Kitani, Karsten Kreis, Sanja Fidler, and Or Litany. Trace and pace: Controllable pedestrian animation via guided trajectory diffusion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

- [9] Mohamed Hassan, Yunrong Guo, Tingwu Wang, Michael J. Black, Sanja Fidler, and Xue Bin Peng. Synthesizing physical character-scene interactions. In *ACM SIGGRAPH 2023 Conference Proceedings*, 2023.
- [10] Jordan Juravsky, Yunrong Guo, Sanja Fidler, and Xue Bin Peng. PADL: Language-directed physics-based character control. In *SIGGRAPH Asia 2022 Conference Papers*, 2022.
- [11] Xue Bin Peng, Yunrong Guo, Lina Halper, Sergey Levine, and Sanja Fidler. ASE: Large-scale reusable adversarial skill embeddings for physically simulated characters. *ACM Transactions on Graphics (Proc. SIGGRAPH)*, 41(4), 2022.
- [12] Zhengyi Luo, Jinkun Cao, Josh Merel, Alexander Winkler, Jing Huang, Kris Kitani, and Weipeng Xu. Universal humanoid motion representations for physics-based control. In *International Conference on Learning Representations (ICLR)*, 2024.
- [13] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th International Conference on Machine Learning (ICML)*, 2009.
- [14] Martin de Lasa, Igor Mordatch, and Aaron Hertzmann. Feature-based locomotion controllers. *ACM Transactions on Graphics (Proc. SIGGRAPH)*, 29(4), 2010.
- [15] Thomas Geijtenbeek, Michiel van de Panne, and A. Frank van der Stappen. Flexible muscle-based locomotion for bipedal creatures. *ACM Transactions on Graphics (Proc. SIGGRAPH Asia)*, 32(6), 2013.
- [16] Jehee Lee, Jinxiang Chai, Paul S. A. Reitsma, Jessica K. Hodgins, and Nancy S. Pollard. Interactive control of avatars animated with human motion data. *ACM Transactions on Graphics (Proc. SIGGRAPH)*, 21(3), 2002.
- [17] Libin Liu, KangKang Yin, Michiel van de Panne, Tianjia Shao, and Weiwei Xu. Sampling-based contact-rich motion control. *ACM Transactions on Graphics (Proc. SIGGRAPH)*, 29(4), 2010.
- [18] Jiashun Wang, Huazhe Xu, Jingwei Xu, Sifei Liu, and Xiaolong Wang. Synthesizing long-term 3D human motion and interaction in 3D scenes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [19] Sirui Xu, Zhengyuan Li, Yu-Xiong Wang, and Liang-Yan Gui. InterDiff: Generating 3D human-object interactions with physics-informed diffusion. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [20] Yinhuai Wang, Jing Lin, Ailing Zeng, Zhengyi Luo, Jian Zhang, and Lei Zhang. PhysHOI: Physics-based imitation of dynamic human-object interaction. *arXiv preprint arXiv:2312.04393*, 2023.
- [21] Zeqi Xiao, Tai Wang, Jingbo Wang, Jinkun Cao, Wenwei Zhang, Bo Dai, Dahua Lin, and Jiangmiao Pang. Unified human-scene interaction via prompted chain-of-contacts. In *International Conference on Learning Representations (ICLR)*, 2024.
- [22] Abhinanda R. Punnakal, Arjun Chandrasekaran, Nikos Athanasiou, Alejandra Quiros-Ramirez, and Michael J. Black. BABEL: Bodies, action and behavior with english labels. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [23] Mathis Petrovich, Michael J. Black, and Gül Varol. Action-conditioned 3D human motion synthesis with transformer VAE. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [24] Guy Tevet, Sigal Raab, Brian Gordon, Yonatan Shafir, Daniel Cohen-Or, and Amit H. Bermano. Human motion diffusion model. In *International Conference on Learning Representations (ICLR)*, 2023.
- [25] Jingbo Wang, Zhengyi Luo, Ye Yuan, Yixuan Li, and Bo Dai. PACER+: On-demand pedestrian animation controller in driving scenarios. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [26] Chen Tessler, Yoni Kasten, Yunrong Guo, Shie Mannor, Gal Chechik, and Xue Bin Peng. CALM: Conditional adversarial latent models for directable virtual characters. In *ACM SIGGRAPH 2023 Conference Proceedings*, 2023.

- [27] Zhiyang Dou, Xuelin Chen, Qingnan Fan, Taku Komura, and Wenping Wang. C-ase: Learning conditional adversarial skill embeddings for physics-based characters. In *SIGGRAPH Asia 2023 Conference Papers*, 2023.
- [28] NVIDIA. ProtoMotions: Physics-based character animation. <https://github.com/NVlabs/ProtoMotions>, 2024.
- [29] Mohamed Hassan, Duygu Ceylan, Ruben Villegas, Jun Saito, Jimei Yang, Yi Zhou, and Michael J. Black. Stochastic scene-aware motion prediction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [30] Matthew Loper, Naureen Mahmood, Javier Romero, Gerard Pons-Moll, and Michael J. Black. SMPL: A skinned multi-person linear model. *ACM Transactions on Graphics (Proc. SIGGRAPH Asia)*, 34(6), 2015.

A Implementation Details and Reproduction

Architecture The implementation extends the ProtoMotions codebase [28] with the adaptive masking policy. The fully constrained controller π_{FC} is a transformer-based PPO policy (6 layers, 8 attention heads) initialized from the pretrained ProtoMotions full-body tracker. The partially constrained controller π_{PC} is distilled with DAgger using a transformer inpainting model that consumes 120 past steps and predicts 2 future frames. The masking policy π_θ is a two-layer MLP with 128 hidden units, trained with PPO (learning rate 10^{-4} , discount $\gamma = 0.99$), and outputs $\rho \in [0.1, 0.9]$ from the Δ validation loss, mask entropy, and gradient norm. The reward is Eq. 4 with $\alpha = 1.0$ and $\beta = 0.5$ on imitation accuracy. The scheduler is integrated into `train_agent.py`; checkpoints are saved every 10,000 steps and logged metrics include MSE, accuracy, ρ , and motion smoothness. The change spans approximately 400 lines across the modified files. Figure 8 shows the mask-generation code path.

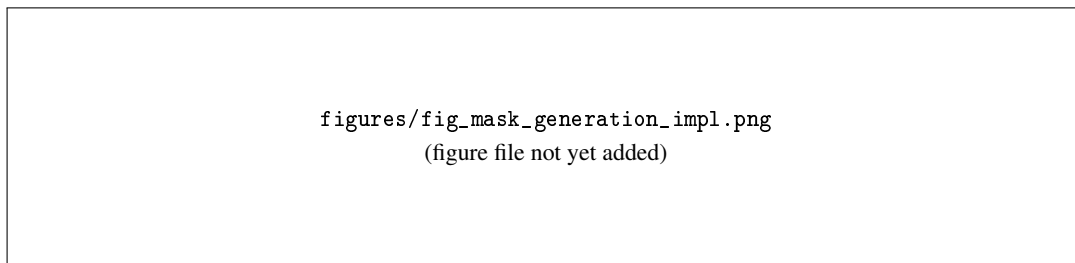


Figure 8: Mask-generation implementation.

Setup

```
git clone https://github.com/NVlabs/ProtoMotions
cd ProtoMotions
pip install -e .
pip install -r requirements_isaacgym.txt
git lfs fetch --all          # pretrained checkpoints
```

Set the AMASS dataset path in `config/base.yaml`. IsaacGym is configured for 128 parallel environments on a single NVIDIA A100.

Data preparation Load AMASS, preprocess into 120-step sequences with 2 future frames, and split 80%/10%/10% into train/validation/test.

Training

```
python train_agent.py +exp=maskedmimic
```

Train for 2,000,000 steps: Phase 1 (0–1M steps) distills π_{PC} with fixed $\rho = 0.5$; Phase 2 (1M–2M steps) trains π_{PC} and π_θ jointly with adaptive masking.

Evaluation

```
python eval_agent.py +checkpoint=checkpoint_2000000.pt
```

Computes MSE, imitation accuracy, mask entropy, joint-velocity smoothness, and mask-ratio convergence rate. Generalization on unseen cartwheel sequences is recorded with `record_video=true`.

Reference numbers Values a successful run should reproduce (single seed):

- ρ converges to ≈ 0.68 by about 1.8M steps; convergence rate (95% of final mean) ≈ 1.5 M steps.
- Mask entropy stabilizes near 0.8; joint-velocity smoothness ≈ 0.02 .
- Cartwheel hold-out: MSE $\approx 0.18 \rightarrow 0.105$ and MPJPE $45 \rightarrow 39$ mm from the Phase 1 checkpoint to the adaptively trained model.
- Ablation without π_θ : MSE ≈ 0.14 and smoothness ≈ 0.04 , about 25% higher error than the full system.